From Machine-Checked Runs to Machine-Checked Proofs: A Formalization Plan for Papers 1 and 2

Status: Tiers 1 and 2 delivered. The module Proofs.idr (940 lines, %default total) typechecks with zero errors and compiles under Idris2 0.8.0; because the module is total, the clean build is the proof. Section 6 records the Tier 1 delivery and Section 7 the Tier 2 delivery, each with its deviations from the plan as drafted.

Concept note v0.3, Projects secret1 and secret2, August 11, 2026

1. What changes, and why it matters

Every quantitative claim in both papers is machine-checked in the weak sense: an Idris2 program computes the claim's instances and prints OK. The type system has so far been used as a programming language, not as a logic. This plan upgrades the strongest claims to the strong sense: theorems whose statements are Idris2 types and whose proofs are total terms, checked once by the compiler and never re-run.

The claims fall into three kinds, and each kind formalizes differently.

Schematic reduction facts. Statements of the form 'for all terms u and v, this term reduces to that term'. Examples: the pointwise validity of R1 to R4, the identity behavior of S K x, the fusion fact pair a b X reduces to a (X b). These become universally quantified theorems over an inductive reduction relation, proved by exhibiting reduction chains plus a substitution lemma. Fully provable.

Arithmetic invariants along inductions. Statements of the form 'for all n and k, this process computes that value in this many steps'. Examples: Theorem A, the probe correctness of Proposition 1, the division certificates of Proposition 5. These become inductions over the numeral structure, with a kit of modular arithmetic lemmas doing the bookkeeping. Provable with grind.

Finite claims. Statements over a stated finite domain: the census, the null results, the E5 windows. These are not universally quantified theorems and should not pretend to be; they become verified computations, a decidable checker plus a generic lemma lifting 'the checker returned True on the enumerated domain' to the bounded Pi-statement. The existing sweeps become proofs of exactly what they always were: exhaustive checks with stated bounds.

One discipline change carries all of this: the proof modules are `%default total`, while the experiment engine stays `covering`. The codebase splits into a small package (Term and reduction relations; the engine; the proofs) so the totality boundary is clean.

2. The theorem inventory, paper 1

2.1 Theorem A: primality is grafting-indecomposability

Claim. For n at least 2: n is prime exactly when no binary tree with n leaves is decomposable as a graft of two trees each with at least 2 leaves.

Current evidence. Census computation to n = 12.

Formal route. The entire theorem reduces to one structural lemma plus elementary number theory:

```
leavesGraft : (s, t : LTree) -> leaves (graft s t) = leaves s * leaves t
```

Proof by induction on s; the Node case is distributivity. Then decomposable-at-n implies n = a * b with a, b at least 2 (not prime), and composite n implies a decomposable witness by grafting any a-tree onto any b-tree (existence of trees at every size is a one-line induction). The Prime predicate and the fact that a product of two numbers at least 2 is not prime are standard Nat reasoning.

Tier 1. This is the flagship easy proof and should be done first.

2.2 Adequacy of the unary numerals
Claim. For all n, the numeral applied to f and x weakly reduces to f applied n times to x.

Formal route. With Steps the reflexive-transitive contextual closure of one-step weak reduction:

```
numAdequate : (n : Nat) ->
  Steps (App (App (num n) (Var "f")) (Var "x")) (fpow n (Var "x"))
```

Induction on n; the base and step are short concrete chains (KI f x to x; SB m f x through B to f applied to the recursive case) glued by congruence constructors. **Tier 1**, and the ideal warm-up for the reduction-relation machinery.

2.3 Pointwise validity of R1 to R4
Claim. Each rule's two sides, applied to fresh f and z, are weakly joinable, for all parameter terms u and v.

Formal route. Two ingredients. First, a substitution lemma, immediate because Var has no reduction rule:

```
substSteps : (sig : String -> Term) ->
  Steps s t -> Steps (subst sig s) (subst sig t)
```

Second, the concrete joins at Var-instantiated parameters, produced mechanically by a certified evaluator (Section 4.2). Together they yield the universally quantified statements, one per rule:

```
r1Valid : (u, v : Term) ->
  Joinable (App (App (r1L u v) (Var "f")) (Var "z"))
           (App (App (r1R u v) (Var "f")) (Var "z"))
```

Tier 1. This turns the papers' 'machine-verified validity' into theorems.

2.4 Theorem B, fragment form: reachable boxes are exact factorizations
Claim as formalized. For n at least 2: n is composite exactly when there exist m, k at least 2 with the product box over their numerals R-reducing to the numeral of n.

Formal route. Two lemmas, and notably no confluence is needed.

```
boxReduces : (m, k : Nat) ->
  StepsR (prodBox (num m) (num k)) (num (m * k))

stepRPreservesVal : StepR t t' -> fragVal t = fragVal t'
```

The first is the schematic forward derivation, by induction on m with R1 to R4 chains and congruence; it also delivers the cost formula as a by-product if steps are counted in the relation. The second is a semantic invariant: a partial denotation on the arithmetic fragment (combs to their length, boxes to products, sum boxes to sums), preserved by every rule; then any R-derivation from a box to a numeral forces m * k = n by injectivity of the denotation on numerals. Composite implies reachable is `boxReduces` at any factorization; reachable implies composite is the invariant. The original paper proved unreachability via confluence and exhaustive search; the invariant route is strictly cleaner for formalization, and the confluence framing remains as commentary. **Tier 2** (the induction

in boxReduces is real work; the invariant needs care about which contexts the fragment denotation is congruent for, which is why the theorem is stated for the fragment).

2.5 What stays open from paper 1

Strong normalization of R_arith was honestly open in the paper and remains so; nothing here changes that. Local confluence via critical pairs is formalizable in principle but is a project of its own (Tier 3); the specific critical-pair joins can be proved as concrete lemmas without the general theorem.

3. The theorem inventory, paper 2

3.1 Proposition 1: probe correctness and cost

Claim. For all n, k at least 2, the divisibility probe reaches the residue comb of n mod k in exactly bitlen n rule applications, each preserving the affine invariant.

Formal route. Define a view mirroring the binary recursion, so induction follows the numeral's shape:

```
data BinV : Nat -> Type where
  VOne  : BinV 1
  VEven : {m : Nat} -> BinV m -> BinV (2 * m)
  VOdd  : {m : Nat} -> BinV m -> BinV (2 * m + 1)

probeCorrect : (k : Nat) -> 2 `LTE` k -> (v : BinV n) ->
  RunProbe k n = (mod n k, bitLen n)
```

The inductive steps are the two identities r + p(2m) = r + (2p)m and r + p(2m + 1) = (r + p) + (2p)m taken mod k, which land in the modular lemma kit (Section 4.4). **Tier 2** the mathematics is trivial, the mod-lemma bookkeeping is the grind.

3.2 Proposition 2: primality is F-irreducibility

```
fIrreducible : (n : Nat) -> 2 `LTE` n ->
  (splitMoves (num n) = []) <-> Prime n
```

Reduces to: a numeral has exactly one F-position (structural), and the divisor filter over 2 to n minus 1 is empty exactly when n is prime (Nat reasoning with a Dec-based Prime). **Tier 1.**

3.3 Proposition 3: termination of F_arith

Formal route. The paper's multiset argument simplifies: the measure mu t, the sum over leaves of (value minus 1), strictly decreases under every split, because for v = k * q with k, q at least 2, (v minus 1) minus (k minus 1) minus (q minus 1) equals (k minus 1)(q minus 1), which is positive. Sorting preserves mu and strictly decreases the inversion count. So:

```
splitDecreases : DoSplitStep t t' -> mu t' `LT` mu t
sortDecreases  : SortStep t t'   -> (mu t' = mu t, inv t' `LT` inv t)
fArithTerminates : (t : FForm) -> Accessible FStep t
```

The first two are Tier 1 or low Tier 2; packaging them into well-founded accessibility over the lexicographic pair (mu, inversions) uses Idris2' s Accessible machinery and is **Tier 2**. Note the pleasant contrast this cements formally: the factoring orientation terminates provably while the multiplying orientation's termination stays open.

3.4 Proposition 4: confluence is unique factorization

Formal route, layered honestly. Fully provable now: the semantic invariant (every F-step preserves the product of leaf values) and the normal-form characterization (F-normal means every leaf prime, right-nested, sorted):

```
fStepPreservesVal : FStep t t' -> fval t = fval t'
nfCharacter : FNormal t -> SortedPrimeChain t
```

These reduce uniqueness of normal forms to: two sorted lists of primes with equal product are equal, which is the fundamental theorem of arithmetic. The equivalence theorem is then stated conditionally, `uniqueNF : FTA -> ...`, with FTA either imported or proved as an optional standalone module (strong induction plus least-prime-divisor; a few hundred lines; classical and self-contained). **Tier 2 for the layers, Tier 3 for unconditional.** This layering is itself faithful to the paper's claim that F_arith reformulates FTA rather than reproving it.

3.5 Proposition 5: division certificates

Same shape as 3.1: induction over BinV, invariant k * Q + 2^i * rest = v + 2^i * r, plus the parity lemma that (r + k * q) minus b is even when q is chosen as the parity of r + b and k is odd. The k = 2 case is a single pattern equation. **Tier 2.**

3.6 The E4 lemmas: the small jewels

These are the best effort-to-payoff proofs in the plan.

```
kHeadConstant : (n, z : Term) -> Steps (App (App CK n) z) n
skIdentity    : (x, z : Term) -> Steps (App (App (App CS CK) x) z) z
```

The first is one constructor; the second is a three-step chain, universally quantified over both terms. And the star:

```
ssNeverIterator : (x : Term) -> (n : Nat) -> IsIterP (S n) s ->
  StuckShape (wnfOf (App (App (App (App (App CS CS) x) s) (Var "f")) (Var "x")))
```

with `IsIterP n s` the parametric iterator property (for every argument term w, s f w reduces to f applied n times to w) and `StuckShape` the root pattern App (App (Var f) something) something. The proof needs one new structural lemma, that a Var-headed root with its argument count is preserved by weak reduction (no rule fires at a Var head), and then the concrete chain derived in the campaign. This makes the paper's cutest impossibility fully formal, quantified over all x. **Tier 1.**

3.7 The E5 claims, upgraded from sweeps to schemata

The bundle correctness claims are secretly schematic and should be proved for all n, not just the swept ranges:

```
bundleCoord : (i : Fin (length basis)) -> (n : Nat) ->
  coordAt i (bundleOf basis n) = Just (unary (mod n (index i basis)))

addBCorrect : addB basis (bundleOf basis n) (bundleOf basis m)
            = Just (bundleOf basis (mod (n + m) (prod basis)), length basis)
```

The coordinate lemma is induction on the basis list; the operation lemmas need the compatibility facts mod (n + m) p = mod (mod n p + mod m p) p and mod (mod n M) p = mod n p when p divides M, from the lemma kit. The primality window claim formalizes with one number-theoretic lemma, that a composite below p squared has a prime factor at most p (if n = a * b then the smaller factor squared is at most n), plus a finite decidable check that the basis lists exactly the primes below its bound. **Tier 2.** The fusion fact pair a b X to a (X b) is a Tier 1 four-step chain.

3.8 What stays outside the type system

Observation 1 (the conversion barrier) is a statement about polynomial-time reductions between problems; formalizing it would require a cost model and a complexity theory library and is out of scope; it remains a proof in prose. The automata obstructions (Minsky and Papert, Hartmanis and

Shank, Shallit) are cited external theorems; formalizing them is a separate project in automata theory. The census and the null results are finite claims and are handled as verified computations (Section 4.5), which is their correct epistemic status.

4. The proof infrastructure

4.1 Inductive reduction relations

The semantic bedrock, replacing 'the evaluator returned this' with 'there exists a derivation':

```
data Step : Term -> Term -> Type where
  RedK : Step (App (App CK a) b) a
  RedS : Step (App (App (App CS a) b) c) (App (App a c) (App b c))
  CtxL : Step l l' -> Step (App l r) (App l' r)
  CtxR : Step r r' -> Step (App l r) (App l r')

data Steps : Term -> Term -> Type where
  Done : Steps t t
  Then : Step s t -> Steps t u -> Steps s u
```

StepR extends Step with four constructors whose indices are the exact R1 to R4 shapes (the concrete subterms sb, skb, ki appear directly in the constructor types). FStep likewise for the factoring rules. Everything downstream is stated against these relations, so the fueled evaluators become untrusted oracles rather than the definition of truth.

4.2 The certified evaluator: industrializing chain proofs

Dozens of lemmas are concrete finite reduction chains. Writing Then constructors by hand does not scale; instead the existing evaluator gets a certified twin:

```
wnfCert : (fuel : Nat) -> (t : Term) -> Maybe (t' ** Steps t t')
```

returning the normal form together with its derivation. A concrete lemma is then proved by running wnfCert inside the proof and closing the goal with decEq on the result. This is proof by reflection in its simplest form, and it converts the campaign's entire verification style into proof-producing style with one function.

4.3 The substitution lemma

`substSteps` (Section 2.3) is what upgrades Var-instantiated chains to universally quantified theorems. It is a structural induction whose key case is vacuous: Step has no rule for Var, so substitution cannot create or destroy redexes at substituted positions. This single lemma is the license behind every 'for all terms u, v' in the plan.

4.4 The modular arithmetic kit

The honest grind. Idris2' s base library is thin on mod lemmas, so Propositions 1 and 5 and the E5 operations need a small module, roughly fifteen lemmas: mod distributes over addition and multiplication, mod of mod under divisibility, mod bounds, the parity facts for the Hensel step, and lcm-style compatibility for the bundle modulus. Self-contained, tedious, reusable.

4.5 The finite-exhaustion bridge

One generic lemma turns sweeps into theorems about bounded domains:

```
allBelowSound : (p : Nat -> Bool) -> (b : Nat) ->
  allBelow p b = True -> (n : Nat) -> n `LT` b -> p n = True
```

The census, the null screening results, and the E5 window sweeps are then stated as what they are, decidable checks over enumerated domains with the bound in the type, and the compiler replays them. No false universality is claimed, and none is needed.

4.6 Module layout and totality boundary

Split Secret2.idr into a package: `Term.idr` (syntax, Eq, Ord), `Reduce.idr` (relations, certified evaluator, substitution lemma), `Engine.idr` (the experiment machinery, covering), `ProofsP1.idr` and `ProofsP2.idr` (total). The bootstrap toolchain needs nothing beyond base; a plain ipkg suffices.

5. Order of work

Tier 1, the first campaign. Reduction relations and Steps; numAdequate; leavesGraft and Theorem A end to end; substSteps; wnfCert and the R1 to R4 validity theorems; kHeadConstant, skIdentity, ssNeverIterator with the stuck-root lemma; the fusion fact; fStepPreservesVal; splitDecreases; fIrreducible. Each is small; together they formalize the two papers' most quotable sentences.

Tier 2, the second campaign. The mod kit; probeCorrect; division certificates; boxReduces and the fragment Theorem B; bundle coordinate and operation correctness; the E5 primality window with the least-factor lemma; termination packaging via Accessible; the finite-exhaustion bridge applied to the census and windows.

Tier 3, open-ended. A standalone FTA module discharging the hypothesis of Proposition 4; critical-pair joins toward local confluence of R_arith; anything touching strong normalization of R_arith, which remains open.

The payoff, stated in the project's own terms: paper 1' s thesis was that the equality does the work, and paper 2 moved the costs into normalization. This plan moves the *evidence* into the types, which is where a project living in a dependently typed language always intended it to end up.

6. Delivery record, Tier 1

Delivered in Proofs.idr, a self-contained total module: the inductive relations Step and Steps with congruence lifts and joinability; the substitution lemma substSteps; the certified evaluator wnfCert returning normal forms together with their derivations; numAdequate; the four validity theorems r1Valid to r4Valid, universally quantified in the rule parameters; the E4 lemmas kHeadConstant, skIdentity, and the stuck-shape pair ssReach and ssStuckForever; the fusion fact; Theorem A end to end (leavesGraft, theoremAto, theoremAfrom, and the prime corollaries in both directions); and the F_arith results fStepVal, splitDecreases, and the irreducibility pair primeIrreducible and irreduciblePrime.

Five deviations from the plan as drafted, all documented in the source:

First, Theorem A is formalized constructively as the equivalence Composite n if and only if Decomp n, with primality handled through PrimeN n defined as n at least 2 together with Not (Composite n). This keeps both directions witness-producing and avoids any appeal to decidability.

Second, the S S impossibility is scoped precisely: ssReach shows the canonical reduct has the stuck root shape, fh2Steps shows that shape persists under every further reduction step, and ssStuckForever concludes no continuation reaches an iterator value. The all-strategies statement would additionally need Church-Rosser for weak reduction and is listed under Tier 3.

Third, FForm leaves carry their values directly, and the Split rule is indexed by the factored value (k plus 2) times (q plus 2) written as successors. Value preservation at splits is then definitional (Refl), both factors are at least 2 by construction, and the strict decrease of the split measure rests on two small structural lemmas rather than polynomial rewriting.

Fourth, the bootstrap toolchain's Data.Nat lacks lteTransitive and lteRefl; the module carries its own five-line LTE kit (lteTrans, lteReflx, lteSuccR, kLeM, lePlusLeft).

Fifth, the Term syntax in Proofs.idr is verbatim-identical to the engine's but lives in its own module; unifying the two behind one shared Term module is Tier 2 housekeeping, as anticipated in Section 4.6.

Idris2 lessons worth recording for Tier 2: let-bound names stay opaque in type ascriptions, so proof targets must be written inline or bound with parenthesized ascriptions; constructor implicits are erased unless bound explicitly in the data declaration; and rewrite replaces left side by right side in the goal, confirmed empirically.

7. Delivery record, Tier 2

Tier 2 is delivered in the same module, now 940 lines, still %default total with no postulates and no holes. Four parts landed.

Fragment Theorem B, without confluence. R_arith lives in the module as an inductive relation StepR, one constructor per rule and two for closure under application contexts, with StepsR as its reflexive transitive closure. The forward direction is the schematic derivation boxReduces : (m, k : Nat) -> StepsR (prodBox (num m) (num k)) (num (m * k)), a five-line induction riding on a two-line unfolding lemma for sum boxes. The backward direction realizes the planned value invariant in a cleaner form than drafted: instead of a partial denotation, arithmetic forms are an inductive family AF : Nat -> Term -> Type indexed by their value, with structural numerals and product and sum boxes, so that value preservation is index preservation (stepRAF : AF n t -> StepR t u -> AF n u). Every rule case holds definitionally; the context cases go through inversion lemmas on the rigid spines. Uniqueness (afUnique) closes the argument: a product box of numerals R-reduces to the numeral of n exactly when n = m * k, with the primality corollaries in both directions. No confluence is used anywhere.

Proposition 1 in divmod form. The probe is formalized over the value-indexed binary view BinV : Nat -> Type, least significant bit outermost, with plus-only index algebra. Both registers stay strictly below k, so each update needs at most one restoring subtraction, certified by red : (k, x : Nat) -> LT x (k + k) -> (r ** d ** (LT r k, r + d * k = x)). The run probeGo threads the affine invariant r + p * rest = r′ + q * k through both bit cases, discharged by two ring identities and two regrouping lemmas, and the wrapper delivers probeCert : (k : Nat) -> LTE 2 k -> BinV n -> (r ** q ** (LT r k, n = r + q * k)). This is the divmod specification: the returned register is the remainder of n by k, quotient attached as certificate. The cost claim is structural rather than embedded: the recursion consumes exactly one view constructor per step, so the run length is bitLen n by construction; a mirrored step counter inside the sigma type was judged noise and dropped, a deviation from the plan's phrasing but not from its content. No modulus function is defined anywhere; the divmod specification replaced the planned modular arithmetic lemma kit, which shrank to eight Nat identities.

Split termination as accessibility. The strict decrease of Tier 1 is packaged as well-foundedness: splitTerminates : (t : FForm) -> Accessible SplitRev t, by strong induction on the leaf measure. Every sequence of splits from any factored form terminates, in the standard constructive sense.

The finite-exhaustion bridge. allBelow with allBelowSound : allBelow p b = True -> (n : Nat) -> LT n b -> p n = True lifts one verified run over an enumerated domain to the bounded universal statement, the schema under which the census and window claims of both papers can be restated as theorems relative to a checked run.

Deferred, with reasons. Proposition 5′ s division certificates follow the probeCert pattern and are deferred as routine rather than hard. Bundle correctness for all n needs the bundle encoder inside the proof module and is deferred with it. Termination of full F_arith, with Swap and Assoc, needs the lexicographic measure and stays with the critical-pair work in Tier 3. Unifying the engine and the proof module behind one shared term type remains housekeeping. The construction of BinV n for arbitrary n by strong recursion on parity is a small addition and can ride along with any of these.